

„ZPR PWr – Zintegrowany Program Rozwoju Politechniki Wrocławskiej”

Prototypowanie gier. GDD.

Marek Kopel

wideo z wykładu:

https://pwr-edu.zoom.us/rec/share/1W8qxAnDtz-DVoZC_RgtqzLsIFHH8jNDpoxYSEJa7akh0fnr_-hAlvyJNLVbgAEZ.jj8AhP0hmSt8bPDE

Lecture

1. **Silniki gier. Wprowadzenie do Unity. Pierwsza gra 2D.**
2. **Mechanika 2D, animacje.**
3. **Oświetlenie, tekstury, materiały. Pierwsza gra 3D.**
4. Narzędzia wspierające, np. Blender.
5. **Zapis/odczyt danych, komunikacja sieciowa.**
6. **Modelowanie terenu. Generowanie terenu.**
7. **Sztuczna inteligencja w grach.**
8. Virtual Reality, wsparcie VR w Unity.
9. **Optymalizacje w Unity.**
10. Historia i klasyfikacja gier.
11. Prototypowanie gier. GDD.
12. Projektowanie poziomów gier.
13. Projektowanie gier dla różnych platform.
14. Testowanie gier. Test.
15. Test.

Game development

https://en.wikipedia.org/wiki/Video_game_development#Development_process

software development process

game = software with **art, audio, and gameplay**

formal methods ← often overlooked →

- likely to run over budget and time estimates
- likely contain a large number of bugs

Planning ← important for **individual** and **group** projects

game development ← **not suited** for **life cycle methods** (e.g. waterfall model)

→ agile development

- **iterative** prototyping
- depends on feedback and **refinement**
- **gradually** increasing feature set
- most projects do not start with a **clear requirement outline**
- most popular → **Scrum**

Game development cont.

Personal Software Process (PSP)

- another **agile** method
- require additional **staff training** (awareness of project's **planning**)
- more **expensive** and requires **commitment** of team members
- possible extension → **Team** Software Process (self-directing team)

usually → **overlap** of many methods, e.g.:

- asset creation ← waterfall model (requirements are clear)
- gameplay design ← iterative prototyping

Game development stages

Pre-production

1. High concept
2. Pitch
3. Concept
4. Game design document
5. Prototype

Programowanie Gier
(I st. - ten kurs :)

Milestones

1. First playable
2. Alpha
3. Code freeze
4. Beta
5. Code release
6. Gold master

Crunch time

Production

1. Design
2. Programming
3. Level creation
4. Art production
5. Audio production
6. Testing

tylko **ZAJAWKA**
- całość na
Gry Komputerowe
II stopień,
spec. PSI

Post-production

1. Maintenance

Gry Komputerowe
(kurs na II st.)

Pre-production

a.k.a. **design phase**

late stage of pre-production ← a.k.a. “**proof of concept**”, “**technical review**”

planning phase ← focused on **idea and concept** development
and production of initial design **documents**

concept development → produce **clear and easy** to understand **documentation**
(**tasks, schedules and estimates** for development team)
→ called “**production plan**”
- usually not funded by a publisher (but required)

concept documentation (3 stages, each a.k.a. “**game proposal**” ← no naming standard):

- **high concept** ← brief description
- **pitch** ← short summary with game's selling points (why would be profitable)
- **concept** (a.k.a. “**game plan**”) ← more detailed:
high concept, game's genre, gameplay description, features, setting, story,
target audience, hardware platforms, estimated schedule, marketing analysis,
team requirements and risk analysis

Pre-production - GDD

publishers ← may expect game proposals with **playable prototypes**
(demos during “green-light meetings”)

Game design document

- needed before a full-scale production
- most of the material from the initial pitch
- describes the game's concept and major gameplay elements
- may include preliminary sketches of various aspects of the game
- sometimes with functional prototypes of some sections of the game

GDD remains a **living document** throughout the development

← changed weekly or even daily

"requirement capture" ← list of game's needs

Usual GDD structure

Story

Characters

Level/environment design

Gameplay

Art

Sound and Music

User Interface, Game Controls

<https://www.nuclino.com/articles/game-design-document-template>

<https://whimsygames.co/blog/game-design-instructions-examples>

Pre-production - prototype

prototypes of gameplay ideas and features

- ← allow programmers and designers to experiment with different algorithms and usability scenarios
- ← help determine what features the design specifies
- ← often act as a proof of concept or to test ideas (adding, modifying or removing some features)

algorithms and features from prototype **may be ported** to the game

developed quickly (little up-front design)

→ RAD (Rapid application development) tools

GOOD CODERS...



... KNOW WHAT THEY'RE DOING

Game creation system (GCS)

https://en.wikipedia.org/wiki/Game_creation_system

a consumer-targeted **game engine** and a set of **specialized design tools**
(and sometimes a light **scripting language**),
engineered for the **rapid iteration** of **user-derived video games**.

Unlike more developer-oriented game engines, game creation systems promise an **easy entry point for novice or hobbyist** game designers, with often little to **no coding required** for simple behaviors.

Although **initially stigmatized**, all-in-one game creation systems have gained some legitimacy with the central role of

- **Unity**,
- **Construct**,
- **GameMaker: Studio**

in the growth of the **indie game development** community. Currently the **Independent Games Festival** recognizes games produced with similar platforms.

Game creation system (GCS) cont.

https://en.wikipedia.org/wiki/Game_creation_system

Early game creation systems such [...] appeared in the **1980s on home computers**.

In **1990s**, game creation systems for the **IBM PC** shifted both to the **more general** and the **more specific**.

Later in the **mid-2000s**, with the growth of the **World Wide Web** and **social networking**, programs like

- **BlitzBasic** (*Worms*)
- **Multimedia Fusion** (*Five Nights at Freddy's*)

headlined an explosion of interest **both in indie games and in canned** game design software.

Whereas **earlier** game creation systems tend **to err on the side of user friendly interfaces**, 21st-century systems are often distinguished by extensive **scripting languages that attempt to account for every possible user variable**.

GCS Features

Tools

- **Integrated Development Environment**: for managing projects and resources
- **Command Line Interface**: for compiling and debugging games
- **Sprite Editor**: for the editing of animated images
- **Model Editor**: for **3D modeling** purposes
- **Map/Scene Editor**: generally used for object and tile placement

Scripting

The rise of game creation systems also saw a rise in the need for free form **scripting languages with general purpose** use. Some packages, such as Conitec's **Gamestudio**, include a more comprehensive **scripting language under the surface** to allow users more leeway in defining their games' behavior.

GCS Usage

While most of the mainstream and popular game creation systems may be general-purpose, **several exist solely for specific genres.**

- **City games:** City building games
- **First-person shooter:** Pie in the Sky
- **Fighting games:** Fighter Maker
- **Role-playing games:** RPG Maker
- **Visual novels:** Ren'Py
- **MMOs:** Roblox
- **Platform games:** LittleBigPlanet, Super Mario Maker, Super Mario Maker 2, Dreams (video game)

https://en.wikipedia.org/wiki/Game_creation_system#Usage (current ver.)

Level editor

https://en.wikipedia.org/wiki/Level_editor

a.k.a **map**, **campaign** or **scenario editor**

software used to **design levels, maps, campaigns, etc. and virtual worlds** for a video game. An individual involved with the creation of game levels is a **level designer** or **mapper**.

In some cases, the creator of a video game releases an **official level editor** for the game, but other times the **community of fans step in** to fill the void. The level editor **can be integrated** into the game; for example, a track editor for a racing game. Other times (and most often), the editor is a **separate application**, as are most fan-released level editors.

One of the first 3D games which became popular partially due to level editors and fan-made content was **Doom**. The creation of various third-party editors led to the birth of an **online community trading fan-made maps**.

Level editor cont.

https://en.wikipedia.org/wiki/Level_editor

A level editor is often limited to creating **levels for only a certain game engine**. Developing a level editor takes a lot of time and **it is more time and cost efficient to release multiple games using the same engine** instead of developing a new engine and level editor for each game. Level editors **offer some limited scope of content creation**, but in the case of gaming industry solutions the **scope is very large** allowing an entire game to be created without the need for much support from a programming team. To make **larger changes** to a game than simply adding new levels, a **software development kit (SDK)** is sometimes needed.

In the early years of video-gaming, some games came with a utility called a **"construction set"**. This was similar in many ways to a level-editor. Some games used them to create extra levels, whereas others (like the *Shoot-'Em-Up Construction Kit*) used them as a means to create a game rather than be a game in itself.

Shoot-'Em-Up Construction Kit

a.k.a. SEUCK

a game creation system for the Commodore 64, Amiga and Atari ST created by Sensible Software and published by Outlaw (part of Palace Software) in 1987

allows the user to make simple shoot 'em ups by drawing sprites and backgrounds and editing attack patterns. The advertising promoted the Kit with the phrase "By the programmers of *Wizball* and *Parallax*".



<https://www.youtube.com/watch?v=WNIIQjwWo5M>

Enhancing SEUCK @ C64



Kodu Game Lab

https://en.wikipedia.org/wiki/Kodu_Game_Lab

a programming integrated development environment (**IDE**)
by Microsoft's FUSE Labs.

runs on Xbox and Windows
since 2009

a visual programming tool → basic coding with the use of blocks and pictures

Kodu is **different from other projects** in several key ways:

- It avoids typing code by having users construct programs using visual elements **via a game controller**
- Rather than a bitmapped or 2D display, programs are executed in a **3D simulation environment**, similar to *Alice*

used as an educational learning tool

Kodu Game Lab cont.

https://en.wikipedia.org/wiki/Kodu_Game_Lab

Kodu's programming model is **simplified**
can be programmed using a gaming controller or keyboard and mouse

It **dispenses** with most "serious" programming conventions, including **symbolic variables, branching, loops, number and string manipulation, subroutines, polymorphism**, and so on.

This **simplicity** is achieved by situating the programming task in a **largely complete simulation environment**. The user programs the **behaviors of characters in a 3d world**, and programs are expressed in a high-level, sensory paradigm consisting of a **rule-based system or language**, based on **conditions and actions similarly to *AgentSheets***.

Kodu Game Lab cont.

https://en.wikipedia.org/wiki/Kodu_Game_Lab

The typical "hello world" of Kodu is:

see - fruit - move - towards

<condition> <action>

where

- <condition> is: <sensor> [<filter> ...]
- <action> is: <verb> [<modifier> ...]

example: **see - red - fruit - move - towards - quickly**

Many different types of games can be made in Kodu, such as **racing, strategy, RPGs, adventure, platform, puzzle, 1st person shooters, and others.**



a **block-based visual programming language**
and online community **targeted primarily at children**

Users of the site can create online projects using a block-like interface
developed by **the MIT Media Lab**, translated into 70+ languages

taught and used in **after-school centers, schools, and colleges, as well as
other public knowledge institutions**

takes its **name** from a technique used by disk jockeys called "scratching", where
vinyl records are clipped together and manipulated on a turntable to produce
different sound effects and music

the website lets users **mix together different media** (including graphics, sound,
and other programs) in creative ways by "remixing" projects



cont.

Category			Notes		
	Motion	Moves sprites, changes angles and position			
	Looks	Controls the visuals of the sprite			
	Sound	Plays audio files and effects			
	Events	Event handlers			
	Control	Conditionals and loops etc.			
Category			Notes		
	Sensing	Sprites can interact with the surroundings			
	Operators	Mathematical operators, comparisons			
	Variables	Variable and List usage and assignment			
	My Blocks	Custom procedures			



cont.

when creating **sprites** and **backgrounds**, users can **draw** their own sprite manually, choose a Sprite from a library, or upload an existing image

Extensions add **extra blocks** and features that can be used in projects: hardware based and software based, such as Text to speech voices

Physical

- **EV3** - control motors and receive sensor data from the LEGO EV3
- **Makey Makey** - use the Makey Makey to control your projects
- **LEGO Education WeDo 2.0** - control motors and receive sensor data from the LEGO WeDo
- **BBC micro:bit** - use of a BBC micro:bit to control projects
- **Lego BOOST** - bring robotic creations to life
- **Go Direct Force & Acceleration** - Sense pull, push, motion, and spin.



cont.

Extensions add **extra blocks** and features that can be used in projects: hardware based and software based, such as Text to speech voices

Digital

many of the digital extensions used to be **regular block categories** that were **moved to the extensions** section **to reduce clutter**. These include:

- Music - Play digital instruments (drums, trumpets, violins, pianos and more)
- Pen - Draw on the Stage with a variety of thicknesses and color
- Video Sensing - Detect motion with the camera.

new digital extensions have also been **added in collaborations with commercial companies**. These include:

- Text to Speech - Converts words in a text into voice output (variety of voices, supplied by [Amazon](#))
- Translate - Uses [Google Translate](#) to translate text from one language into a variety of other languages, including Arabic, Chinese, Dutch, English, French, Greek, and Japanese

users create own extensions for Scratch 3.0 using [JavaScript](#)

Blockly

Blocky Games <https://blockly.games/maze?lang=en&level=10&skin=0>

CODE <https://code.org/>

AppInventor2 <http://ai2.appinventor.mit.edu/#5100454479396864> ball, Space Invaders

GameMaker Studio

https://en.wikipedia.org/wiki/GameMaker_Studio



formerly **Animo** until 1999, **Game Maker** until 2011, **GameMaker** until 2012, and **GameMaker: Studio** until 2017

(<https://en.wikipedia.org/wiki/Undertale>)

a **cross-platform game engine** developed by YoYo Games

accommodates the creation of **cross-platform** and multi-genre **video games**

using a custom drag-and-drop **visual programming language**

or a **scripting language** known as Game Maker Language

(more advanced games that could not be created just by drag and drop)

originally designed to allow **novice computer programmers** to be able to make computer games **without much programming knowledge**

recent versions of software also focus on **appealing to advanced developers**

GameMaker Studio cont.

https://en.wikipedia.org/wiki/GameMaker_Studio



primarily intended for making **games with 2D graphics**, allowing **out-of-box** use of

- **raster graphics**
- **vector graphics** (via **SWF**),
- **2D skeletal animations** (via Esoteric Software's Spine)
- a large standard library for drawing graphics and 2D **primitives**.

software allows for limited use of **3D graphics**, this is in form of **vertex buffer** and **matrix** functions, and as such **not intended for novice users**

The engine uses **Direct3D** on Windows, UWP, and Xbox One;
OpenGL on macOS and Linux;
OpenGL ES on Android and iOS,
WebGL or **2d canvas** on HTML5,
proprietary APIs on **consoles**.

GameMaker Studio cont.

https://en.wikipedia.org/wiki/GameMaker_Studio



The engine's primary element is an **IDE** with built-in editors for raster graphics, **level design**, scripting, **paths**, and **shaders** (GLSL or HLSL).

Additional functionality can be implemented:

- in software's scripting language

- or platform-specific native extensions

In **GameMaker Studio 2**, you can choose whether to export the game as an **NSIS** installer,

- or a **.zip** file containing the game, the data.win file,

- and any files added under the "Included Files" tab in the editor

Garry's Mod

https://en.wikipedia.org/wiki/Garry%27s_Mod



often abbreviated as **GMod**

a **sandbox game** developed by Facepunch Studios
published by **Valve Corporation**

created by Garry Newman as a **mod** for Valve's **Source game engine**
released in December 2004

expanded into a standalone release and published by Valve in November 2006

the base **game mode** of *Garry's Mod* has **no set objectives** and provides the
player with **a world to freely manipulate**

other game modes, notably *Trouble in Terrorist Town*, are created by other
developers as mods and are **installed separately**, by means such as the **Steam Workshop**

See also: **List of video games derived from mods**

Garry's Mod cont.

https://en.wikipedia.org/wiki/Garry%27s_Mod

a physics-based **sandbox game** and its base **game mode** has no set objectives

The player is able to spawn **non-player characters**, **ragdolls**, and **props**, and interact with them by various means.

Using the **physics gun**, ragdolls and props can be picked up, rotated, and frozen in place.

The **tool gun** is a multi-purpose item for various tasks, such as **welding** and constraining props together, and altering the facial expressions of ragdolls.

includes the functionality to modify the game by developing **scripts (Lua programming language)**

Notable mods include *Spacebuild*, *Wiremod*, *Elevator: Source*, *DarkRP*, *Prop Hunt*, and *Trouble in Terrorist Town*.

Specialised servers, known as Fretta servers, rotate between custom game modes every fifteen minutes.



The **player character** (right) positioning characters from *Team Fortress 2* on a couch using the physics gun

DEMO: [posing \(face\)](#),
[NPC spawning \("background NPCs"\)](#)

Game creation system (GCS) cont.

Need a GCS?

What 4?

→ <http://creatools.gameclassification.com>